

FlightGear Knowledge Assistant

28 de mayo del 2026

Asistente de Gestión del Conocimiento para Soporte Técnico en Entorno de Software Militar

Autora: Belén Quintas Herraiz

Programa: Máster IA Generativa 2025/26

Fecha: Mayo 2026

1. Descripción del Proyecto

Contexto y Problema

El equipo de In-Service Support (ISS) del software de simulación de vuelo de aeronaves militares proporciona soporte técnico especializado a clientes cuando surgen incidencias relacionadas con el software: errores, anomalías en cálculos, problemas de datos o dudas funcionales.

La gestión de la información necesaria para atender estas solicitudes enfrenta cuatro retos estructurales:

Fragmentación de la información: Los datos relevantes están distribuidos entre Service Requests en el CRM corporativo, correos electrónicos intercambiados con clientes, bugs y scripts en la plataforma TFS, y documentación técnica en repositorios internos.

Dificultad para centralizar y recuperar información: La dispersión hace que localizar casos similares o soluciones previas sea un proceso lento y dependiente del conocimiento individual de cada ingeniero.

Dependencia del conocimiento tácito: Gran parte de la resolución de problemas depende de la experiencia acumulada de los ingenieros, lo que dificulta la incorporación de nuevo personal y genera inconsistencias en las soluciones.

Repetición de esfuerzos y falta de trazabilidad: La ausencia de un sistema centralizado provoca que problemas ya resueltos vuelvan a analizarse desde cero, aumentando el tiempo de respuesta y el riesgo de errores.

Solución Propuesta

El proyecto plantea un sistema basado en modelos de lenguaje que actúa como asistente interno para ingenieros de ISS, proporcionando acceso inteligente al conocimiento técnico e histórico del software. El sistema no automatiza decisiones críticas ni interactúa directamente con clientes; toda la toma de decisiones finales y la interlocución externa permanece en manos del ingeniero.

La solución se implementa como un agente orquestado en n8n que clasifica las consultas entrantes, adapta dinámicamente la estrategia de recuperación de información según el tipo de consulta, y genera respuestas estructuradas y trazables a partir de fuentes verificadas.

2. Modelos, Configuración y Costes

Estrategia de selección de modelos

La selección de modelos se realizó teniendo en cuenta que el sistema implementa una arquitectura multiagente con responsabilidades diferenciadas. En lugar de utilizar un único modelo para todo el flujo, se optó por asignar distintos modelos según el tipo de tarea, priorizando equilibrio entre capacidad de razonamiento, latencia y coste operativo.

Las principales tareas del sistema son:

- Validación y filtrado de consultas
- Generación de estrategias de recuperación
- Análisis técnico de incidencias
- Recuperación y síntesis documental
- Generación de respuestas finales basadas en contexto RAG

Cada una de estas tareas presenta requisitos distintos en términos de complejidad y precisión, por lo que se seleccionaron modelos específicos según el rol desempeñado dentro del flujo de n8n.

Modelos utilizados en la solución final

Nodo / Rol	Modelo	Justificación
Guardrails	gpt-4.1-nano	Utilizado para validaciones ligeras de seguridad y clasificación de consultas (PII, jailbreak, topical alignment y NSFW). Se priorizó baja latencia y coste reducido.
Strategy Agent (Historical Search)	gpt-4.1-mini	Este agente no resuelve la consulta final, sino que genera una estrategia estructurada de recuperación. Su función consiste en extraer entidades técnicas, identificar síntomas y determinar qué herramientas pueden utilizarse. Se seleccionó un modelo ligero al tratarse principalmente de tareas de planificación y structured extraction.
Strategy Agent (Problem Analysis)	gpt-4o	Aunque este agente no realiza el diagnóstico técnico final, sí debe interpretar consultas potencialmente ambiguas y generar estrategias más complejas de recuperación y análisis. Se priorizó mayor capacidad de comprensión contextual para identificar correctamente componentes afectados, desviaciones y necesidades de comparación entre comportamiento esperado y observado.
Strategy Agent (Documentation Query)	gpt-4.1-mini	Responsable de generar estrategias orientadas exclusivamente a recuperación documental. Al tratarse principalmente de tareas de <i>structured extraction</i> y planificación ligera, se priorizó un modelo eficiente con buena relación coste/capacidad
Answer Agent	gpt-4o	Responsable de generar la respuesta final utilizando el contexto recuperado mediante RAG y respetando las restricciones definidas por los <i>Strategy Agents</i> . Este agente concentra el razonamiento técnico principal del sistema, integrando información procedente de documentación e incidencias históricas para producir respuestas estructuradas, consistentes y contextualizadas. Se priorizó un modelo con mayor capacidad de comprensión contextual y síntesis multi-fuente, especialmente relevante en consultas de análisis técnico y troubleshooting.

Exploración inicial de modelos open-source

Durante las fases iniciales del proyecto se exploraron distintos modelos open-source disponibles en HuggingFace con el objetivo de analizar su posible uso en entornos corporativos on-premise.

Los criterios evaluados fueron:

- Comprensión de documentación técnica
- Capacidad de síntesis multi-fuente
- Consistencia en respuestas técnicas
- Reducción de alucinaciones
- Compatibilidad con despliegues internos

Los modelos analizados fueron:

- DeepSeek V3.1 Terminus
- Qwen3-30B-A3B-Instruct-2507
- openai/gpt-oss-120b

Esta exploración permitió validar posibles alternativas para futuras evoluciones hacia arquitecturas completamente self-hosted, especialmente relevantes en entornos militares o corporativos con restricciones de soberanía del dato.

No obstante, para el prototipo funcional implementado en n8n se optó finalmente por modelos OpenAI debido a:

- Mayor estabilidad operativa
- Integración directa con n8n
- Menor complejidad de despliegue
- Menor esfuerzo de infraestructura durante la fase MVP

Prompts Empleados

El sistema utiliza prompts de sistema diferenciados por cada agente estratégico. Cada uno recibe el tipo de consulta como contexto, extrae entidades relevantes (componente, síntoma, contexto operacional), determina de qué herramienta debe hacer uso y establece la complejidad de análisis. Finalmente devuelve un objeto JSON estructurado.

A modo de ejemplo, el prompt del agente de análisis de problema activa tanto la búsqueda en documentación como en datos históricos y exige comparación entre comportamiento esperado y reportado. El agente de consulta documental, por contraste, activa exclusivamente la búsqueda en el manual de usuario y desactiva explícitamente el acceso a fuentes históricas.

El agente de respuesta final recibe el objeto de estrategia como restricciones vinculantes: si una herramienta no está permitida en la estrategia, no puede invocarse; si la profundidad de análisis es baja, se evita el razonamiento diagnóstico profundo.

Costes Anticipados

Dado que la solución productiva se basa en un modelo de lenguaje open-source desplegado en infraestructura interna, los costes asociados a tokens y licencias comerciales son nulos. Los costes previstos son:

- **Infraestructura:** Servidor para ejecutar los modelos de lenguaje + base de datos vectorial para almacenamiento y búsqueda semántica.
- **Personal y mantenimiento:** Ajuste periódico de prompts y plantillas, actualización de la base de conocimiento con nuevas SRs y documentación, y supervisión de la calidad de las respuestas.

Para el prototipo con modelos de OpenAI, el coste por consulta es bajo dada la naturaleza interna del sistema (volumen moderado de ingenieros, no usuarios finales masivos). El nodo de guardrails usa gpt-4.1-nano deliberadamente para minimizar el coste del filtrado previo.

El mayor consumo computacional se concentra en:

- Generación de embeddings
- Retrieval semántico
- Generación de respuesta final mediante el Answer Agent

En particular, el uso de GPT-4o se reserva para las fases del flujo que requieren mayor capacidad de interpretación y razonamiento. Esto incluye tanto el Strategy Agent de análisis de problemas, encargado de comprender consultas complejas y definir estrategias de recuperación adecuadas, como el Answer Agent, responsable de integrar la información recuperada mediante RAG y generar la respuesta final. Al concentrar los modelos más avanzados únicamente en estas etapas, se consigue equilibrar calidad de respuesta, latencia y coste operativo.

En cuanto al control operativo, se integró Langfuse como plataforma de observabilidad y trazabilidad de ejecuciones. Aunque durante el desarrollo no se logró instrumentar correctamente la captura automática de costes y latencias debido a las limitaciones de integración entre n8n y algunos de los nodos utilizados, la plataforma proporciona capacidades nativas para monitorizar consumo de tokens, costes asociados a modelos y tiempos de respuesta. Esto permitiría, en un entorno productivo, realizar un seguimiento detallado del gasto operativo y detectar posibles oportunidades de optimización en el uso de modelos y recursos computacionales.

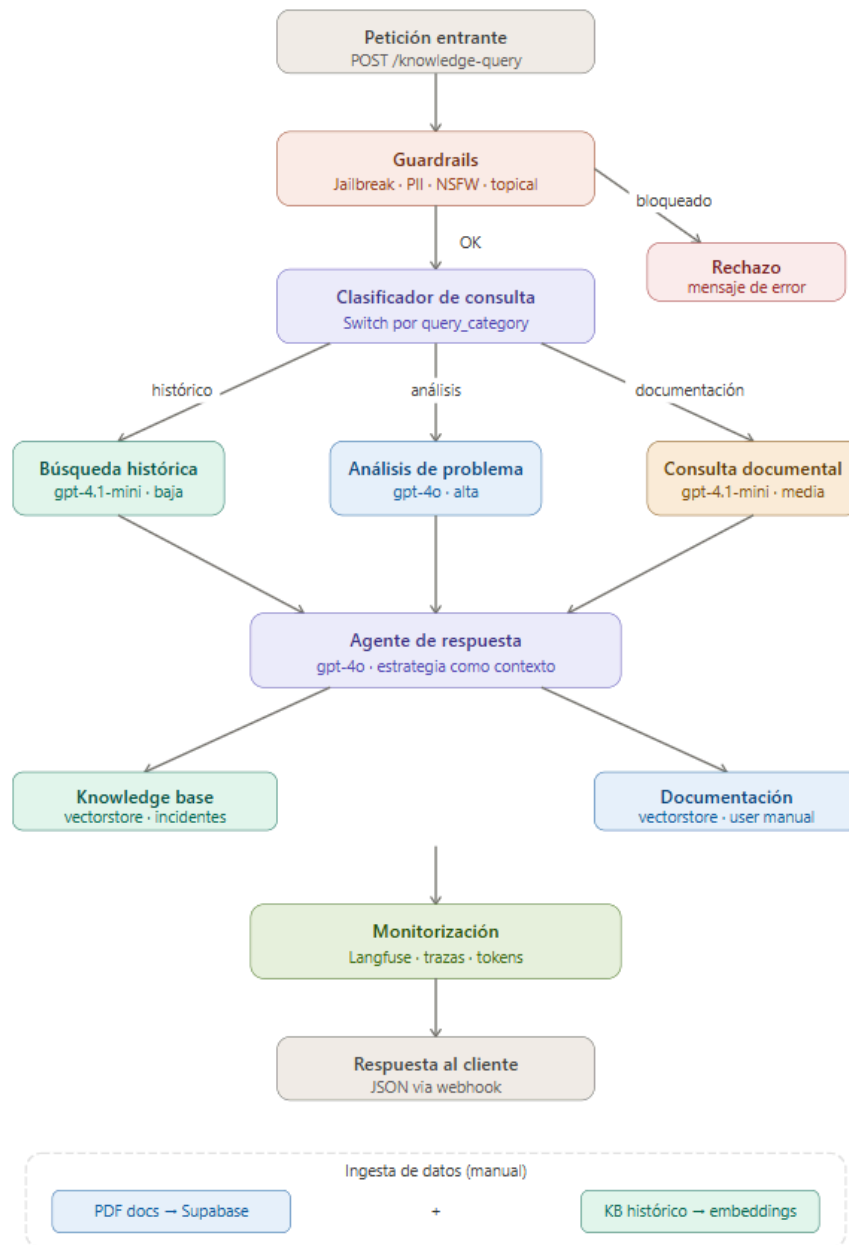
3. Flujo de Agentes y Niveles de Autonomía

Arquitectura Técnica

Componente	Tecnología	Rol
Orquestación	n8n	Motor de flujo y coordinación de agentes
Almacenamiento vectorial	Supabase (pgvector)	Base de conocimiento histórico y documentación técnica
Embeddings	OpenAI text-embedding-3-small	Vectorización de documentos y consultas
Modelos de razonamiento	OpenAI GPT-4o / GPT-4-turbo / GPT-4.1-mini	Agentes estratégicos y de respuesta
Seguridad	n8n Guardrails + HTTP Basic Auth	Filtrado y autenticación
Documentación fuente	Google Drive	Almacenamiento de manuales técnicos (PDF)
Control de versiones	GitHub + VSCode	Gestión del código y configuración

Arquitectura del Flujo

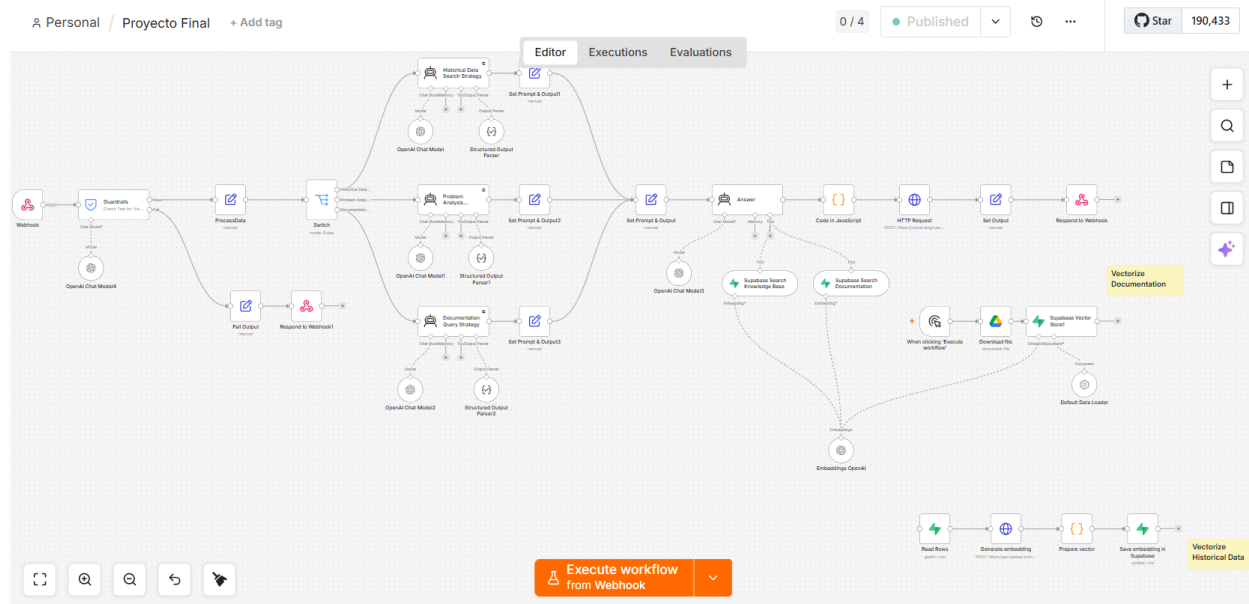
El sistema está implementado como un flujo orquestado en n8n con la siguiente secuencia de procesamiento:



Flujo de Datos

La consulta del ingeniero entra por el webhook, pasa por los guardrails de seguridad, se normaliza y clasifica por tipo, se procesa por el agente estratégico correspondiente (que devuelve un objeto de estrategia estructurado), se enriquece con el prompt de sistema apropiado, y se envía al agente de respuesta final, que invoca las herramientas RAG permitidas y genera la respuesta estructurada que se devuelve al cliente del webhook.

Los flujos de ingestión de conocimiento son independientes y se ejecutan fuera del horario crítico de soporte: la documentación técnica se ingesta descargando PDFs de Google Drive y vectorizándolos en Supabase; los datos históricos se vectorizan incrementalmente procesando solo los registros sin embedding.



Descripción de Nodos y Decisiones de Diseño

Webhook con autenticación básica: El punto de entrada al sistema está protegido con autenticación HTTP Basic Auth, garantizando que solo clientes autorizados puedan enviar consultas.

Nodo Guardrails: Antes de cualquier procesamiento, todas las consultas pasan por un nodo de guardrails que evalúa cuatro dimensiones de seguridad: detección de jailbreak (umbral 0.7), contenido NSFW (umbral 0.9), detección de PII, y alineación temática con el dominio de negocio (soporte técnico FlightGear). Las consultas que no superan el filtro reciben una respuesta de error estandarizada sin llegar al procesamiento principal.

Nodo Process Data: Extrae y normaliza los campos *query* y *query_category* del cuerpo de la petición, desacoplando la lógica de enrutamiento de la estructura del webhook.

Nodo Switch (clasificación por tipo): Enruta la consulta hacia el agente estratégico correspondiente según el valor de *query_category*. Se definen tres rutas: *historical_data_search*, *problem_analysis* y *documentation_query*. Esta separación permite asignar modelos de diferente capacidad y coste según la complejidad requerida por cada tipo.

Agentes de Estrategia (tres nodos): Cada agente extrae entidades técnicas, determina qué fuentes de conocimiento deben consultarse (*allowed_tools*) y establece la profundidad de análisis requerida. Devuelven un objeto JSON estructurado con schema validado por un parser de salida, actuando como

contrato de interfaz hacia el agente de respuesta. Esta capa intermedia es la que implementa la lógica de adaptación dinámica de la estrategia.

Set Prompt & Output (nodos de configuración): Consolidan la salida de la estrategia con el prompt de sistema del agente de respuesta, preparando el contexto completo para la generación final. Cada tipo de consulta tiene un prompt de sistema distinto, optimizado para el tipo de razonamiento requerido.

Agente Answer (respuesta final): Ejecuta la consulta utilizando las herramientas RAG permitidas por la estrategia. Dispone de acceso a dos herramientas vectoriales: *search_user_manual* (documentación técnica en Supabase) y *search_knowledge_base* (base histórica de incidencias, con topK=15 para maximizar la cobertura de casos similares). El agente respeta estrictamente la estrategia: no llama herramientas no autorizadas ni profundiza más de lo indicado.

Nodo Code in JavaScript (preparación de traza): Construye el payload de monitorización destinado a Langfuse. Recoge el input original de la consulta desde el nodo Webhook, el output generado por el agente Answer y la *query_category*.

Nodo HTTP Request (envío a Langfuse): Realiza el POST a la API de ingestión de Langfuse con el batch construido en el nodo anterior, autenticándose mediante las credenciales configuradas. Esta llamada directa a la API es la única vía viable de integración, ya que no existe conector nativo entre n8n y Langfuse. El resultado de este envío no bloquea la respuesta al cliente: el flujo continúa hacia el nodo Set Output independientemente del resultado del registro.

Nodo Set Output y Respond to Webhook: Extraen el output final del agente Answer y lo devuelven al cliente en formato JSON.

Niveles de Autonomía e Intervención Humana

El sistema opera en el nivel más bajo de autonomía agéntica: es un sistema de apoyo a la decisión, no un sistema de acción autónoma. Específicamente:

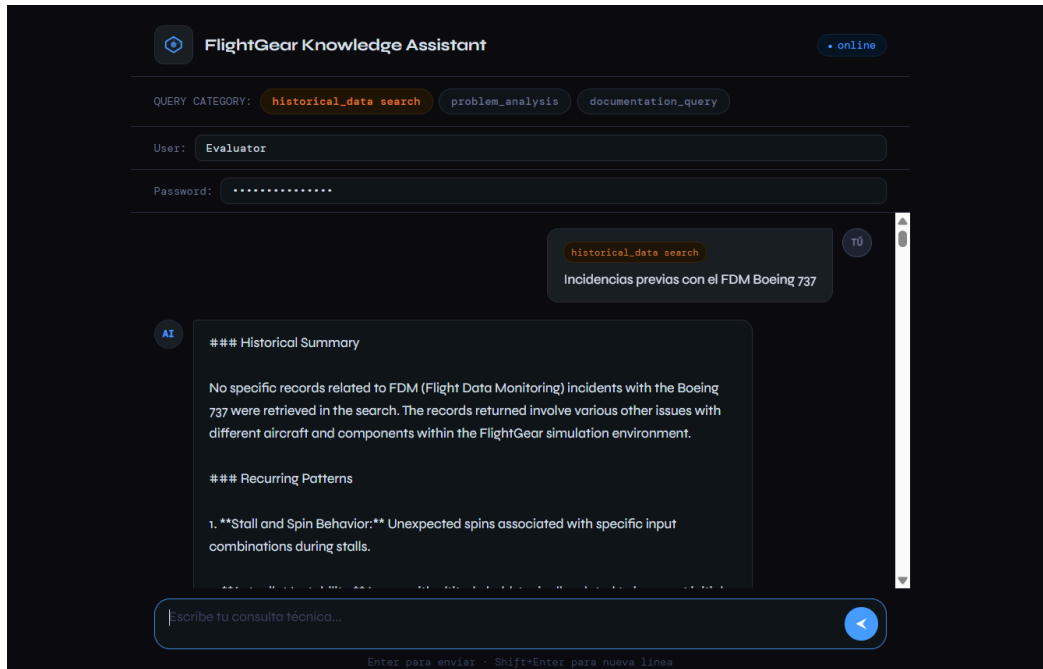
- El ingeniero formula la consulta y decide qué categoría asignarle.
- El agente recupera información y genera una respuesta estructurada con nivel de confianza explícito (0–1).
- El ingeniero revisa y valida la información antes de aplicarla o enviar cualquier comunicación al cliente.
- No se ejecutan scripts automáticamente, no se modifican SRs ni bases de datos sin intervención humana.

Este nivel de control humano es una restricción de diseño derivada del entorno corporativo y militar, donde las decisiones técnicas y las comunicaciones con el cliente son responsabilidad del ingeniero.

4. Despliegue, Evaluación y Monitorización

Estrategia de Despliegue

Para exponer el agente se ha optado por activar el flujo de n8n como webhook HTTP y construir una interfaz web estática que se conecta a él. La interfaz ha sido desarrollada con HTML, CSS y JavaScript vanilla, sin dependencias de frameworks, lo que la hace ligera y fácil de desplegar.



Por qué Vercel

Se ha elegido Vercel por tres razones principales. El despliegue es inmediato conectando directamente el repositorio de GitHub, sin configuración de servidor ni build steps complejos para un sitio estático. Vercel asigna además un dominio público HTTPS de forma automática, lo que permite compartir la URL sin pasos adicionales. Por último, cualquier actualización del código se despliega automáticamente en segundos vía CI/CD, alineándose con las buenas prácticas de integración y entrega continua.

Consideraciones sobre el entorno de despliegue: manejado vs autogestionado

El proyecto se enmarca en un contexto militar donde los requisitos de seguridad y soberanía del dato exigen habitualmente despliegues on-premise o en nubes privadas certificadas, sin dependencia de servicios externos como n8n Cloud, Supabase Cloud o los modelos de OpenAI en su modalidad pública. En un entorno productivo real, la arquitectura debería adaptarse a infraestructura propia: n8n self-hosted, una base de datos vectorial desplegada internamente y modelos LLM en local o en entornos air-gapped.

No obstante, la arquitectura diseñada es suficientemente modular como para migrar cada componente a su equivalente on-premise sin cambios estructurales en el flujo.

La arquitectura actual no incorpora contenedores dado que tanto n8n como Supabase se utilizan en modalidad cloud gestionada. Sin embargo, se ha analizado la viabilidad de una versión contenerizada y la conclusión es que sería técnicamente sencilla, ya que existen imágenes Docker oficiales para todos los componentes del stack: n8nio/n8n para el motor de flujos, ankane/pgvector para PostgreSQL con soporte vectorial, y nginx para servir la interfaz web estática.

Evolución prevista

La migración a un entorno contenerizado con Docker Compose no implicaría cambios en la lógica del agente ni en el flujo de n8n, sino únicamente en la capa de infraestructura. Migrar el flujo de n8n Cloud a una instancia self-hosted, reconectar todas las credenciales y verificar el comportamiento equivalente queda identificado como una evolución pendiente, relevante de cara al despliegue on-premise que requeriría el entorno militar real para el que está concebido el sistema.

Evaluación

La validación del sistema se abordó desde dos perspectivas complementarias. Por un lado, se realizó una evaluación offline mediante un conjunto de pruebas controladas con DeepEval para medir la calidad del sistema antes de su despliegue. Por otro lado, se incorporaron mecanismos de monitorización en producción mediante Langfuse con el objetivo de proporcionar trazabilidad y observabilidad sobre las ejecuciones reales del sistema.

Evaluación offline

La evaluación offline se llevó a cabo utilizando la librería DeepEval, ampliamente utilizada para la validación de aplicaciones basadas en modelos de lenguaje y arquitecturas RAG.

El objetivo de esta fase fue analizar el comportamiento del sistema frente a un conjunto de escenarios representativos de los tres tipos de consultas soportadas.

Para ello se diseñó un banco de pruebas compuesto por nueve casos de evaluación que cubrían tanto escenarios positivos como consultas con información incompleta o potencialmente fuera del alcance de la base de conocimiento.

Métricas evaluadas

La evaluación se realizó utilizando las siguientes métricas:

Métrica	Descripción
Faithfulness	Mide el grado en que la respuesta está respaldada por el contexto recuperado.
Answer Relevancy	Evalúa la alineación entre la respuesta generada y la consulta realizada.
Contextual Precision	Mide la relevancia de los fragmentos recuperados por el sistema de retrieval.
Contextual Recall	Evalúa la capacidad de recuperar toda la información necesaria para responder correctamente.

Resultados obtenidos

Los resultados agregados obtenidos fueron los siguientes:

Métrica	Score medio	Pass Rate
Faithfulness	0.95	100%
Answer Relevancy	0.88	88.9%
Contextual Precision	0.45	22.2%
Contextual Recall	0.81	88.9%

Los resultados muestran un comportamiento especialmente sólido en términos de fidelidad al contexto recuperado, obteniendo una puntuación media de 0.95 y una tasa de éxito del 100%. Esto indica que el sistema genera respuestas mayoritariamente fundamentadas en la información disponible y presenta una baja tendencia a introducir contenido no respaldado por las fuentes recuperadas.

Asimismo, las métricas de relevancia y recall presentan valores elevados, lo que demuestra que el sistema es capaz de localizar información útil y generar respuestas alineadas con las necesidades del usuario.

Sin embargo, la métrica de Contextual Precision obtuvo resultados significativamente inferiores. El análisis detallado de los casos evaluados mostró que, aunque la información relevante suele recuperarse correctamente, frecuentemente aparece mezclada con documentos o fragmentos menos relevantes en posiciones superiores del ranking. Esta situación afecta negativamente a la precisión del retrieval y constituye actualmente la principal limitación técnica identificada en la solución.

Principales conclusiones

La evaluación realizada permite extraer varias conclusiones:

- El sistema responde correctamente a la mayoría de consultas técnicas evaluadas.
- La arquitectura de agentes y el uso de estrategias especializadas contribuyen a mantener una elevada fidelidad de las respuestas.
- El principal cuello de botella no se encuentra en la generación de respuestas, sino en la fase de recuperación y ranking de contexto.
- Futuras mejoras deberían centrarse en técnicas de re-ranking, búsqueda híbrida y optimización de retrieval para incrementar la precisión contextual.

En conjunto, los resultados obtenidos validan la viabilidad de la arquitectura propuesta y demuestran que el sistema es capaz de proporcionar respuestas fiables para escenarios de soporte técnico aeronáutico.

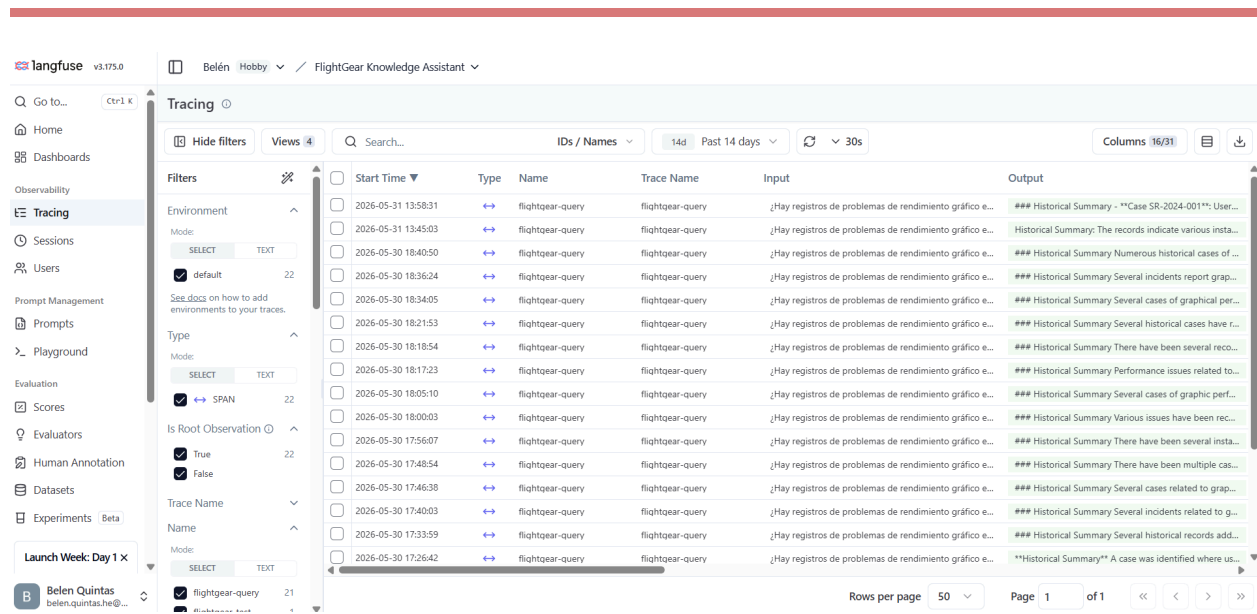
Monitorización en producción

Además de la evaluación offline, se incorporó una capa de observabilidad mediante Langfuse con el objetivo de facilitar el seguimiento de las ejecuciones realizadas por el sistema.

La integración permite registrar información relevante de cada interacción, incluyendo:

- Consulta recibida por el asistente.
- Categoría de consulta seleccionada.
- Respuesta final generada.
- Identificador de ejecución asociado a cada interacción.

Esta instrumentación proporciona un nivel inicial de trazabilidad que permite revisar las consultas procesadas y las respuestas generadas durante las pruebas, facilitando tareas de depuración y análisis del comportamiento general del sistema.



Durante el desarrollo se intentó ampliar la monitorización incorporando métricas adicionales como consumo de tokens, costes de inferencia y latencias de ejecución. Sin embargo, debido a las limitaciones encontradas en la integración entre Langfuse y determinados componentes utilizados en n8n, no fue posible instrumentar estas métricas de forma fiable dentro del alcance temporal del proyecto.

Evolución futura

Como trabajo futuro se plantea ampliar la monitorización incorporando indicadores adicionales como:

- Coste por consulta.
- Latencia media por agente.
- Distribución de tiempos de respuesta.
- Tasa de consultas bloqueadas por guardrails.
- Métricas de calidad basadas en feedback de usuarios.

La combinación de evaluación offline y monitorización en producción proporciona una visión integral del comportamiento del sistema, permitiendo evaluar tanto su calidad funcional como su comportamiento operativo una vez desplegado.

5. Seguridad y Análisis de Riesgos

Durante el diseño del sistema se realizó un análisis de riesgos orientado a identificar las principales amenazas asociadas al uso de agentes, recuperación semántica y exposición del servicio mediante interfaces web y webhooks.

A partir de este análisis se implementaron diversos mecanismos de mitigación que permiten reducir significativamente la superficie de ataque del sistema.

Riesgos asociados a modelos de lenguaje

Uno de los principales riesgos de cualquier aplicación basada en LLMs es la posibilidad de que el modelo genere respuestas incorrectas, incompletas o no respaldadas por las fuentes de información disponibles.

Este fenómeno, conocido habitualmente como *hallucination*, puede provocar que el usuario reciba información técnicamente incorrecta o interprete como válida una respuesta generada por el modelo sin suficiente evidencia documental.

Para reducir este riesgo, la solución utiliza una arquitectura Retrieval Augmented Generation (RAG) en la que las respuestas se generan a partir de información recuperada previamente desde la documentación técnica y la base histórica de incidencias. Además, se incorporó un proceso de evaluación offline mediante DeepEval para medir métricas de fidelidad y relevancia de las respuestas.

No obstante, el sistema debe entenderse como una herramienta de apoyo a la toma de decisiones y no como un sustituto de la validación técnica realizada por especialistas.

Prompt Injection y Jailbreak

Las arquitecturas basadas en agentes son especialmente vulnerables a intentos de manipulación mediante técnicas de prompt injection y jailbreak.

Un usuario podría intentar introducir instrucciones destinadas a modificar el comportamiento esperado del sistema, acceder a información no relacionada con el dominio funcional o provocar respuestas fuera del alcance previsto.

Para mitigar este riesgo se implementó una capa de guardrails situada inmediatamente después del webhook de entrada y antes del routing interno de agentes.

Los controles configurados incluyen:

- Detección de intentos de jailbreak.

-
- Filtrado de contenido inapropiado (NSFW).
 - Detección de información personal identificable (PII).
 - Validación de alineamiento temático (*topical alignment*).

El control de alineamiento temático restringe el uso del sistema a consultas relacionadas con soporte técnico, incidencias históricas, diagnóstico de problemas y documentación de FlightGear, bloqueando automáticamente peticiones fuera de este ámbito.

Las consultas que no superan estas validaciones son rechazadas antes de llegar a los agentes de procesamiento.

Riesgos específicos de arquitecturas RAG

Además de los riesgos tradicionales asociados a los LLMs, las arquitecturas RAG introducen amenazas adicionales relacionadas con las fuentes de conocimiento utilizadas por el sistema.

Una de las más relevantes es el denominado *indirect prompt injection*, donde instrucciones maliciosas podrían almacenarse dentro de documentos indexados en la base vectorial y posteriormente ser recuperadas durante el proceso de búsqueda.

Aunque el prototipo utiliza documentación controlada y una base histórica gestionada internamente, este riesgo debe considerarse en futuros despliegues productivos.

Como medida de mitigación, la arquitectura implementa una separación explícita entre:

- Consulta del usuario.
- Estrategia generada por los agentes.
- Contexto recuperado.
- Instrucciones del sistema.

Además, los agentes de estrategia restringen las herramientas que pueden utilizarse y generan instrucciones estructuradas para el agente final, reduciendo la probabilidad de que información recuperada modifique el comportamiento general del sistema.

Protección de datos

El contexto real de este proyecto corresponde a un entorno militar, donde parte de la información gestionada puede considerarse sensible o clasificada. Por este motivo, se incorporó un mecanismo de detección de información personal identificable (PII) dentro de la capa de guardrails.

Asimismo, debe considerarse que las consultas son procesadas mediante servicios externos de inferencia, por lo que cualquier despliegue productivo requeriría una revisión detallada de aspectos relacionados con:

- Clasificación de la información procesada.

-
- Transferencia internacional de datos.
 - Cumplimiento de requisitos corporativos de seguridad.

Como evolución futura se contempla la migración hacia modelos open-source desplegados internamente para incrementar el control sobre los datos procesados.

Control de acceso

El sistema se expone mediante un webhook accesible desde una interfaz web desplegada en Vercel.

Para evitar accesos no autorizados se incorporó autenticación HTTP Basic Auth en el punto de entrada del flujo, limitando el uso del asistente a usuarios autorizados.

Esta medida reduce el riesgo de utilización indebida del servicio y ayuda a controlar el consumo asociado a las llamadas a modelos externos.

No obstante, en un entorno corporativo sería recomendable complementar este mecanismo mediante sistemas de identidad centralizados, control granular de permisos y auditoría completa de accesos.

Riesgos tecnológicos y operativos

La arquitectura implementada depende de múltiples servicios externos, incluyendo:

- OpenAI
- Supabase
- Vercel
- n8n
- Langfuse

La indisponibilidad de cualquiera de estos componentes puede afectar al funcionamiento global del sistema.

Asimismo, el uso continuado de modelos comerciales introduce costes variables asociados al volumen de consultas y al consumo de tokens.

Para mejorar la observabilidad se integró Langfuse como mecanismo de monitorización de ejecuciones, permitiendo registrar consultas, categorías y respuestas generadas durante las pruebas. Esta integración constituye una primera aproximación a la trazabilidad operativa del sistema, aunque futuras versiones deberían ampliar las capacidades de auditoría, métricas de rendimiento y control de costes.

6. Conclusiones y Trabajo Futuro

El proyecto implementa un sistema de asistencia al conocimiento para ingenieros de soporte técnico aeronáutico que aborda los cuatro retos identificados: centralización de la información, recuperación semántica eficiente, reducción de la dependencia del conocimiento tácito individual, y trazabilidad de las soluciones.

La arquitectura de agentes con estrategia adaptativa permite asignar el modelo y la profundidad de razonamiento adecuados a cada tipo de consulta, optimizando tanto la calidad de las respuestas como el coste operativo. Los controles de seguridad multicapa y el principio de control humano en el bucle garantizan que el sistema puede operar en un entorno corporativo y militar con datos sensibles.

Como líneas de evolución futura, se plantea la posibilidad de sustituir progresivamente los modelos comerciales utilizados por alternativas open-source desplegadas internamente, reduciendo la dependencia de proveedores externos y aumentando el control sobre los datos procesados. En esta misma línea, la arquitectura podría migrar hacia un entorno completamente on-premise, incluyendo modelos, bases vectoriales y herramientas de monitorización, facilitando su adopción en organizaciones con requisitos elevados de seguridad y soberanía del dato. Asimismo, sería recomendable ampliar las capacidades de evaluación y observabilidad incorporando métricas más avanzadas relacionadas con latencia, costes operativos, calidad del retrieval, precisión del routing entre agentes y evolución del rendimiento a lo largo del tiempo, permitiendo una gestión más eficiente y basada en datos del sistema en entornos productivos.